

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks: 50

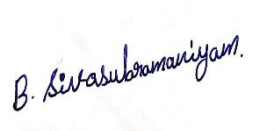
Experiment Questions:

Code of Conduct:

I **SIVA SUBRAMANIYAM B/192572089**(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



B. Sivasubramaniyam.

COURSE OUTCOME COVERED

C03: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

QUESTIONS: 5

Total Marks: 50

Experiment Questions:

- 1.Design and implement a program to simulate the translation of a logical address into a physical address using base and limit registers. The program should accept logical address as input, check for validity, and compute the corresponding physical address. Display appropriate messages for valid and invalid addresses.
- 2.Write a program to demonstrate the paging memory management scheme. The program should divide memory into fixed-size pages and frames, accept page number and offset as input, and compute the corresponding physical address using a page table.
- 3.Develop a program to simulate a page table. The program should allow the user to input page table entries and logical addresses, then map them to physical addresses. Clearly show how page numbers are translated into frame numbers.
- 4.Implement a program to simulate the working of a Translation Lookaside Buffer (TLB). The program should demonstrate how TLB improves memory access time by calculating the effective memory access time with and without TLB.
- 5.Write a program to illustrate memory protection using base and limit registers. The program should verify whether a given logical address lies within the valid range and prevent unauthorized access.
6. Develop a program to simulate dynamic storage allocation using First Fit, Best Fit, and Worst Fit strategies. The program should accept the sizes of available memory blocks and process requests, allocate memory using each strategy, and display the allocated blocks and remaining unused memory.
7. Write a program to simulate swapping between main memory and secondary storage. The program should accept the memory size, process sizes, and process arrival/removal requests, and display which processes are swapped in and swapped out at each stage.
8. Implement a program to simulate segmentation memory management. The program should accept segment numbers and offsets along with a segment table containing base and limit values. It should validate the offset and calculate the corresponding physical address for valid memory references.
9. Develop a program to demonstrate segmentation with paging. The program should accept a logical address consisting of segment number, page number, and offset, use the segment table and page table for translation, and display the final physical address or an appropriate error message.

10. Write a program to simulate virtual memory using demand paging. The program should accept a sequence of page references and the number of available frames, identify page faults, load required pages into memory, and display the contents of each frame after every reference.
11. Implement a program to compare FIFO, LRU, and Optimal page replacement algorithms. The program should accept a page-reference string and number of available frames, simulate all three algorithms, display the frame contents after each reference, and calculate the total number of page faults for each algorithm.
12. Develop a program to simulate thrashing and working-set management. The program should accept a page-reference sequence and working-set window size, determine the pages currently required by a process, calculate the working-set size at different intervals, and identify situations where excessive page faults may cause thrashing.
13. Write a program to simulate dynamic linking. The program should maintain a symbol table containing functions from dynamically linked libraries and allow a program to request a function at execution time. The program should determine whether the required symbol is available and display the corresponding library and memory address.
14. Implement a program to simulate a linker using a simplified two-pass approach. The program should accept multiple object modules containing symbols and references, construct an external symbol table during Pass 1, resolve external references during Pass 2, and display the final linked memory addresses.
15. Develop a program to simulate disk scheduling algorithms such as FCFS, SSTF, SCAN, and C-SCAN. The program should accept the initial disk-head position and a sequence of disk requests, calculate the head movement for each algorithm, display the order in which requests are serviced, and determine the total head movement.

RUBRICS FOR CALCULATION:

Criteria	Weightage (Marks)	Level 4 - Exemplary (Full Marks)	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Conceptual Understanding	35 Marks	Demonstrates clear, complete, and in-depth understanding of OS concepts (types, structure, system calls)	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answers	25 Marks	All answers are correct, precise, and relevant	Mostly correct with minor errors	Partially correct with noticeable errors	Mostly incorrect or irrelevant
Application of Knowledge	15 Marks	Effectively applies concepts to scenarios/questions	Applies with minor mistakes	Limited or partial application	No meaningful application
Completeness of Response	15 Marks	All parts of each question are fully answered	Most parts covered with small omissions	Some parts missing	Incomplete answers
Clarity & Presentation	10 Marks	Clear, well-structured, and neatly presented answers	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 int main() {
3     int base, limit, logical, physical;
4     printf("Enter Base Register: ");
5     scanf("%d", &base);
6     printf("Enter Limit Register: ");
7     scanf("%d", &limit);
8     printf("Enter Logical Address: ");
9     scanf("%d", &logical);
10    if (logical >= 0 && logical < limit) {
11        physical = base + logical;
12        printf("Valid Address\n");
13        printf("Physical Address = %d\n", physical);
14    } else {
15        printf("Invalid Logical Address\n");
16    }
17    return 0;
18 }
```

Run

Output

Enter Base Register: 1000
Enter Limit Register: 500
Enter Logical Address: 250
Valid Address
Physical Address = 1250

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int pageTable[10];
5     int page, offset, frame, physical;
6
7     printf("Enter page table entries:\n");
8     for (int i = 0; i < 10; i++)
9         scanf("%d", &pageTable[i]);
10
11     printf("Enter Page Number: ");
12     scanf("%d", &page);
13
14     printf("Enter Offset: ");
15     scanf("%d", &offset);
16
17     frame = pageTable[page];
18     physical = frame * 100 + offset;
19
20     printf("Frame Number = %d\n", frame);
21     printf("Physical Address = %d\n", physical);
22
23     return 0;
24 }
```

Run

Output

Enter page table entries:
3
5
7
2
8
1
6
9
4
0

Enter Page Number: 4
Enter Offset: 35
Frame Number = 8
Physical Address = 835

=== Code Execution Successful ===

Stream

Online C Compiler - Programiz

Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz

C Online Compiler

Programiz PRO

main.c

Share

Run

```
1 #include <stdio.h>
2
3 int main() {
4     int pageTable[10] = {5, 2, 7, 1, 4, 6, 3, 8, 9, 0};
5     int page = 3;
6     int offset = 25;
7
8     printf("Page Table:\n");
9     for(int i = 0; i < 10; i++) {
10         printf("Page %d -> Frame %d\n", i, pageTable[i]);
11     }
12
13     printf("\nPage Number = %d\n", page);
14     printf("Offset = %d\n", offset);
15
16     printf("Frame Number = %d\n", pageTable[page]);
17     printf("Physical Address = %d\n", pageTable[page] * 100 +
18         offset);
19
20     return 0;
}
```

Output

Clear

Page Table:
Page 0 -> Frame 5
Page 1 -> Frame 2
Page 2 -> Frame 7
Page 3 -> Frame 1
Page 4 -> Frame 4
Page 5 -> Frame 6
Page 6 -> Frame 3
Page 7 -> Frame 8
Page 8 -> Frame 9
Page 9 -> Frame 0

Page Number = 3
Offset = 25
Frame Number = 1
Physical Address = 125

=== Code Execution Successful ===

Stream

Online C Compiler - Programiz

Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz

C Online Compiler

Programiz PRO

main.c

Share

Run

```
1 #include <stdio.h>
2
3 int main() {
4     float memoryAccess = 100;
5     float tlbAccess = 20;
6     float hitRatio = 0.8;
7
8     float emat, withoutTLB;
9
10    emat = hitRatio * (tlbAccess + memoryAccess) +
11        (1 - hitRatio) * (tlbAccess + 2 * memoryAccess);
12
13    withoutTLB = 2 * memoryAccess;
14
15    printf("Memory Access Time = %.0f ns\n", memoryAccess);
16    printf("TLB Access Time = %.0f ns\n", tlbAccess);
17    printf("TLB Hit Ratio = %.1f\n", hitRatio);
18
19    printf("\nEffective Memory Access Time (EMAT) = %.2f ns\n",
20        emat);
21    printf("Memory Access Time without TLB = %.2f ns\n",
22        withoutTLB);
23
24    return 0;
}
```

Output

Clear

Memory Access Time = 100 ns
TLB Access Time = 20 ns
TLB Hit Ratio = 0.8

Effective Memory Access Time (EMAT) = 140.00 ns
Memory Access Time without TLB = 200.00 ns

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int base = 1000;
5     int limit = 500;
6     int logicalAddress = 300;
7     int physicalAddress;
8
9     printf("Base Register = %d\n", base);
10    printf("Limit Register = %d\n", limit);
11    printf("Logical Address = %d\n", logicalAddress);
12
13    if (logicalAddress >= 0 && logicalAddress < limit) {
14        physicalAddress = base + logicalAddress;
15        printf("\nAccess Granted\n");
16        printf("Physical Address = %d\n", physicalAddress);
17    } else {
18        printf("\nAccess Denied\n");
19    }
20
21    return 0;
22 }
```

Output

Base Register = 1000
Limit Register = 500
Logical Address = 300

Access Granted
Physical Address = 1300

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int blockSize[5] = {100, 500, 200, 300, 600};
5     int processSize[4] = {212, 417, 112, 426};
6
7     int nb = 5, np = 4;
8     int i, j;
9
10    printf("Memory Blocks:\n");
11    for(i = 0; i < nb; i++)
12        printf("Block %d = %d\n", i + 1, blockSize[i]);
13
14    printf("\nProcess Sizes:\n");
15    for(i = 0; i < np; i++)
16        printf("Process %d = %d\n", i + 1, processSize[i]);
17
18    printf("\nFirst Fit Allocation:\n");
19
20    for(i = 0; i < np; i++) {
21        for(j = 0; j < nb; j++) {
22            if(blockSize[j] >= processSize[i]) {
23                printf("Process %d allocated to Block %d\n", i
24                    + 1, j + 1);
25                blockSize[j] -= processSize[i];
26            }
27        }
28    }
```

Output

Memory Blocks:
Block 1 = 100
Block 2 = 500
Block 3 = 200
Block 4 = 300
Block 5 = 600

Process Sizes:
Process 1 = 212
Process 2 = 417
Process 3 = 112
Process 4 = 426

First Fit Allocation:
Process 1 allocated to Block 2
Process 2 allocated to Block 5
Process 3 allocated to Block 2
Process 4 cannot be allocated

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int memory = 1024;
5     int process = 512;
6
7     printf("Memory Size = %d KB\n", memory);
8     printf("Process Size = %d KB\n", process);
9
10    if (process <= memory) {
11        printf("\nProcess Swapped In\n");
12        printf("Process Executed\n");
13        printf("Process Swapped Out\n");
14    } else {
15        printf("\nNot Enough Memory\n");
16    }
17
18    return 0;
19 }
```

Run

Output

Memory Size = 1024 KB
Process Size = 512 KB

Process Swapped In
Process Executed
Process Swapped Out

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int base[3] = {1000, 2000, 3000};
5     int limit[3] = {300, 400, 500};
6
7     int segment = 1;
8     int offset = 200;
9
10    printf("Segment Table:\n");
11    printf("Segment\tBase\tLimit\n");
12
13    for(int i = 0; i < 3; i++) {
14        printf("%d\t%d\t%d\n", i, base[i], limit[i]);
15    }
16
17    printf("\nSegment Number = %d\n", segment);
18    printf("Offset = %d\n", offset);
19
20    if(offset < limit[segment]) {
21        printf("Valid Address\n");
22        printf("Physical Address = %d\n", base[segment] + offset);
23    } else {
24        printf("Invalid Offset\n");
25    }
26 }
```

Run

Output

Segment Table:
Segment Base Limit
0 1000 300
1 2000 400
2 3000 500

Segment Number = 1
Offset = 200
Valid Address
Physical Address = 2200

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int pageTable[3][3] = {
5         {5, 6, 7},
6         {2, 3, 4},
7         {8, 9, 10}
8     };
9
10    int segment = 1;
11    int page = 2;
12    int offset = 50;
13    int frameSize = 100;
14
15    printf("Segment = %d\n", segment);
16    printf("Page = %d\n", page);
17    printf("Offset = %d\n", offset);
18
19    printf("\nPage Table:\n");
20    printf("Segment 0 : 5 6 7\n");
21    printf("Segment 1 : 2 3 4\n");
22    printf("Segment 2 : 8 9 10\n");
23
24    int frame = pageTable[segment][page];
25    int physicalAddress = frame * frameSize + offset;
```

Output

Segment = 1
Page = 2
Offset = 50

Page Table:
Segment 0 : 5 6 7
Segment 1 : 2 3 4
Segment 2 : 8 9 10

Frame Number = 4
Physical Address = 450

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int pages[10] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};
5     int frames[3] = {-1, -1, -1};
6
7     int n = 10;
8     int f = 3;
9     int i, j, k = 0;
10    int pageFaults = 0;
11    int found;
12
13    printf("Page Reference String:\n");
14
15    for(i = 0; i < n; i++) {
16        printf("%d ", pages[i]);
17    }
18
19    printf("\n\nFrame Status:\n");
20
21    for(i = 0; i < n; i++) {
22        found = 0;
23
24        for(j = 0; j < f; j++) {
25            if(frames[j] == pages[i]) {
```

Page Reference String:
7 0 1 2 0 3 0 4 2 3

Frame Status:
After page 7: 7 -1 -1
After page 0: 7 0 -1
After page 1: 7 0 1
After page 2: 2 0 1
After page 0: 2 0 1
After page 3: 2 3 1
After page 0: 2 3 0
After page 4: 4 3 0
After page 2: 4 2 0
After page 3: 4 2 3

Total Page Faults = 9

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main() {
4     int pages[10] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};
5     int frames[3] = {-1, -1, -1};
6
7     int n = 10;
8     int f = 3;
9     int i, j, k = 0;
10    int pageFaults = 0;
11    int found;
12
13    printf("Page Reference String:\n");
14    for(i = 0; i < n; i++)
15        printf("%d ", pages[i]);
16
17    printf("\n\nFIFO Page Replacement:\n");
18
19    for(i = 0; i < n; i++) {
20
21        found = 0;
22
23        for(j = 0; j < f; j++) {
24            if(frames[j] == pages[i]) {
25                found = 1;
```

Output

Page Reference String:
7 0 1 2 0 3 0 4 2 3

FIFO Page Replacement:
After page 7: 7 -1 -1
After page 0: 7 0 -1
After page 1: 7 0 1
After page 2: 2 0 1
After page 0: 2 0 1
After page 3: 2 3 1
After page 0: 2 3 0
After page 4: 4 3 0
After page 2: 4 2 0
After page 3: 4 2 3

Total Page Faults = 9

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 int main() {
3     int pages[8] = {1, 2, 3, 2, 4, 1, 5, 2};
4     int n = 8;
5     int window = 3;
6     printf("Page Reference String:\n");
7     for(int i = 0; i < n; i++)
8         printf("%d ", pages[i]);
9     printf("\n\nWorking Sets:\n");
10    for(int i = 0; i < n; i++) {
11        int start = i - window + 1;
12        if(start < 0)
13            start = 0;
14        printf("{ ");
15        for(int j = start; j <= i; j++) {
16            printf("%d ", pages[j]);
17        }
18        printf("\n");
19    }
20    return 0;
21 }
```

Output

Page Reference String:
1 2 3 2 4 1 5 2

Working Sets:
{ 1 }
{ 1 2 }
{ 1 2 3 }
{ 2 3 2 }
{ 3 2 4 }
{ 2 4 1 }
{ 4 1 5 }
{ 1 5 2 }

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2 #include <string.h>
3 int main(){
4     char symbol[]="printf";
5     printf("Requested Function: %s\n",symbol);
6     if(strcmp(symbol,"printf")==0){
7         printf("Library: stdio.h\n");
8         printf("Address: 1000\n");
9     }
10    else if(strcmp(symbol,"scanf")==0){
11        printf("Library: stdio.h\n");
12        printf("Address: 1010\n");
13    }
14    else{
15        printf("Symbol Not Found\n");
16    }
17    return 0;
18 }
```

Run

Output

Requested Function: printf
Library: stdio.h
Address: 1000

=== Code Execution Successful ===

Stream Online C Compiler - Programiz Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz
C Online Compiler

Programiz PRO

main.c

```
1 #include <stdio.h>
2
3 int main(){
4     int baseAddress=2000;
5     int offset=350;
6     int finalAddress;
7
8     printf("Base Address: %d\n",baseAddress);
9     printf("Offset: %d\n",offset);
10
11     finalAddress=baseAddress+offset;
12
13     printf("Final Linked Address = %d\n",finalAddress);
14
15     return 0;
16 }
```

Run

Output

Base Address: 2000
Offset: 350
Final Linked Address = 2350

=== Code Execution Successful ===

CO3 AT1 High Experiments - Cl xCSA04 OS_CO3_AT_High.docx xOnline C Compiler - Programiz x+

Ask Gemini

programiz.com/c-programming/online-compiler/

Programiz

C Online Compiler

Programiz PRO

main.c

ShareRun

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(){
4     int requests[5]={98,183,37,122,14};
5     int n=5;
6     int head=53;
7     int movement=0;
8     printf("Disk Requests:\n");
9     for(int i=0;i<n;i++)
10         printf("%d ",requests[i]);
11     printf("\nInitial Head Position: %d\n",head);
12     for(int i=0;i<n;i++){
13         movement += abs(head-requests[i]);
14         head=requests[i];
15     }
16     printf("\nTotal Head Movement = %d\n",movement);
17     return 0;
18 }
```

Output

Clear

Disk Requests:
98 183 37 122 14
Initial Head Position: 53

Total Head Movement = 469

=== Code Execution Successful ===